大模型辅助的面向对象软件分析、设计与开发

1 课程大作业说明

本大作业围绕真实软件系统开展性能优化、需求扩展或模块开发，完成系统理解、需求分析与建模、核心流程与对象设计、方案实现及效果验证，形成系统分析与设计报告，并提交可运行的代码。所有方向均需建立在对对象、职责、关系和接口的理解之上，基于面向对象方法完成分析、设计与实现。

1.1 选题与完成方式

以**个人**为单位，不组队。学生可选择课程推荐项目中的一个具体方向，或自荐本人参与的研究所科研实践任务。自荐项目应采用**面向对象语言**，目标系统源码规模达到**万行级**，选题具体到本学期能够完成的开发任务。每位学生应确定**一个**具体方向作为本学期大作业选题，每人限选**一个**方向。

为避免选题**过度集中**，并保证各方向的指导与课堂研讨质量，课程将根据报名情况适当平衡各方向人数。若某一方向报名人数过多，原则上按照报名**先后顺序**确定，授课团队将联系较晚报名的学生，协商调剂至其他方向。

1.2 大模型辅助与基本要求

允许使用大模型及 Code Agent 辅助代码理解、需求分析、方案设计、实现与测试。学生应结合源码、运行结果和实验验证模型建议，修正与实际情况的偏差，对最终提交的代码、报告及设计决策负责。

作业应说明具体问题或需求、相关对象与接口、方案选择依据和实际效果。需提供配套测试，记录开发中发生的方案迭代、未采用的思路及原因，并能够在课堂研讨中解释自己的分析、设计和实现。

1.3 阶段安排

第 2 周：授课团队发布项目与方向候选清单。

第 3 周：学生确定项目及具体方向。

第 8、9 周：（1）提交**报告初版**，利用大模型辅助理解目标系统的代码结构、核心对象与执行流程，结合具体任务开展问题或需求分析、对象建模与初步方案设计；对大模型生成的分析结果进行验证和修正，形成自己的工程判断。（2）第一次**课堂研讨**，围绕问题或需求分析与初步方案进行 PPT 分享，提交系统分析与设计报告初版。

第 13 周：提交**报告第二版**，形成初步实现。记录方案迭代过程，包括尝试过的不同方案、失败的思路及其原因，并结合实际结果对原有判断进行修正。

第 15—18 周：**结题研讨**，整体汇报系统理解、设计、实现与验证结果。

第 18 周：提交**报告最终版及最终代码**。

1.4 提交与评分

提交报告、代码、必要的运行说明与测试结果，并按学号安排参加两次课堂研讨。材料提交至 Gitee 等源码托管平台的个人仓库。评分关注报告质量、实现能否正常运行、配套测试与实际效果，以及课堂研讨表现；其中课堂研讨占比 35%。

Mandol 智能体记忆系统

2 Mandol 项目

Mandol 面向大模型智能体的记忆组织、存储与检索。本项目侧重需求驱动的系统扩展：学生需要理解现有记忆模型、数据结构和相关接口，分析新需求与已有架构之间的差距，选择合理位置完成扩展，并验证设计与实现的效果。

仓库地址：https://github.com/AgentCombo/Mandol

指导教师：许利杰研究员

联系人：张宇涵 zhangyuhan25@otcaix.iscas.ac.cn

2.1 AI Coding 增强

本方向聚焦面向 Code Agent 的记忆数据结构。代码开发中的记忆涉及代码实体、Issue、修改过程、测试结果和失败尝试等信息，需要支持“某个函数曾出现什么问题”“某次修改为什么被撤销”等访问需求。学生应分析现有记忆表示对这类场景的支持程度，扩展相应的数据结构、关联关系与访问接口。

主要挑战在于如何划分领域对象、关联开发过程与结果，以及处理代码版本变化和历史信息的追溯。要求基于 Mandol 完成可运行的扩展，通过典型代码开发场景验证其有效性，并说明对象设计及接口选择的依据。无需开发完整的 Code Agent。

2.2 融合索引增强

记忆访问可能同时包含语义相似、精确标识、属性约束和关系查询等需求，单一访问方式或简单组合未必能有效支持复杂场景。学生应分析 Mandol 现有索引与检索机制，针对具体需求，扩展多种访问条件或检索方式的协同能力。可以围绕现有索引进行优化、增加新的索引或检索方式，也可以探索更复杂的组合查询，例如“语义相似 + 属性约束 + 关系约束”，支持查询“与给定内容语义相近、发生于某时间之后，并且属于某实体邻居的记忆”。

主要挑战在于如何组织不同查询条件、协调候选结果，以及保证新增能力与现有检索流程的良好衔接。要求完成与现有系统衔接的实现，并比较检索效果、访问开销及维护成本。具体采用索引优化、新增索引、结构融合、协同查询或其他方案，由学生依据分析和实验确定。

2.3 超长记忆增强

本方向聚焦长期运行场景下的记忆内容管理。随着智能体持续运行，记忆不断积累，系统需要兼顾资源开销、检索效率与有效信息的保留。学生应分析 Mandol 现有记忆组织方式，针对长期记忆中的冗余、重复或低价值信息，设计并实现相应的管理策略。

主要挑战在于如何判断不同记忆的信息价值，以及权衡记忆规模、检索效果和资源开销。要求基于现有系统完成扩展，并通过实验验证所采用策略的效果与局限。具体方案由学生确定，可以探索记忆筛选、合并、摘要或其他长期记忆管理方式。

Ripple 消息引擎

3 Ripple 项目

Ripple 项目侧重已有系统的性能优化。学生需要从实际运行现象出发，理解相关对象、数据组织和执行流程，定位问题、提出优化方案，并验证修改后的正确性与实际效果。

仓库地址：https://github.com/ISCAS-SSG/Ripple

指导教师：唐震副研究员

联系人：唐震　tangzhen12@otcaix.iscas.ac.cn

3.1 内存占用优化

Ripple 在大规模集群和高负载下内存占用较高，限制了集群的扩展性。请分析 Ripple 的内存占用和当前的集群规模上限，设计实现合适的机制优化 Ripple 的内存占用，在同等硬件资源条件下提升 Ripple 可管理的集群最大规模。

3.2 CPU 占用优化

Ripple 现有的多轮次推送模式在特定场景下仍然存在 CPU 使用率较高的问题。请分析 Ripple 在推送数据时的性能边界，针对 Ripple 当前数据推送拓扑不适用的场景扩展合适的数据推送策略，在同等硬件资源条件下降低 CPU 占用，提升 Ripple 的数据推送吞吐率。

3.3 网络带宽优化

Ripple 适用于网络条件较好的数据中心，直接用于边缘侧延迟较高、吞吐率受限、连接不稳定的弱网环境时存在性能问题。请分析 Ripple 的网络带宽占用，设计实现合适的机制降低 Ripple 在数据同步时的网络数据包大小，提升其在边缘弱网环境中的性能。

ChaosBlade 混沌工程

4 ChaosBlade 项目简介

ChaosBlade 项目围绕智能化混沌生成开展模块开发。学生依据给定需求和接口约束，自主完成对象建模、职责划分、核心流程设计与模块实现，并与课程提供的其他模块对接。

仓库地址：https://github.com/chaosblade-io/chaosblade-space-exploration

指导教师：吴国全研究员

联系人：郑明卓 zhengmingzhuo@otcaix.iscas.ac.cn

4.1 基于事故报告的缺陷知识提取

当前混沌实验主要依靠人工指定故障节点、故障类型和参数，缺少来自真实事故的缺陷知识支撑。厂商事故复盘、开源社区缺陷报告和相关文献中包含大量真实案例，但这些材料以自然语言为主，格式不统一，事实、推断和缺失信息相互交织，难以被平台直接使用。本方向要求将事故报告转化为结构统一、来源可追溯的缺陷知识。模块以事故报告的 URL 或正文为输入，提取相关实体、失效过程、缺陷模式、成立条件和激活扰动，形成可供检索、审核和下游模块使用的知识条目。对于多份报告中反复出现的同类机制，应区分具体事故实例和可迁移的缺陷模板；无法从原文确认的信息须标记为推断或缺失，不得由大模型自行补全。

主要挑战在于如何从不完整、表述不一致的事故材料中形成稳定的领域模型，并保持知识的可追溯性。学生需要围绕事故报告、缺陷实例、缺陷模板、证据和抽取任务等概念合理划分对象职责，设计对象之间的关系及对外接口，并处理多阶段任务、重复执行和历史版本等问题。要求基于 Java 21 和 Spring Boot 3.5.5 实现可独立部署的服务，与平台已有的大模型客户端和故障类型字典对接，完成事故报告提交、知识抽取、结果查询和模板检索等核心功能。使用不少于 10 份公开事故报告进行验证，形成不少于 15 条缺陷模板，并通过人工对照评价抽取结果的准确性和可追溯性。

4.2 基于异构系统的缺陷迁移

缺陷知识库中的模板通常来源于其他组织的生产系统，而不同系统在服务命名、技术栈、配置方式和运行指标上存在明显差异，因此无法直接依据表层名称或原事故参数完成迁移。平台现有实验主要检查指标是否越过阈值，难以进一步判断被测系统是否存在某类已知缺陷，也无法区分“系统不存在该缺陷”和“当前扰动未能触发该缺陷”。本方向要求将缺陷模板与被测系统的拓扑、配置和代码信息进行匹配，识别可能存在的风险位置，并根据目标系统自身条件生成可执行的混沌实验。模块需要形成系统画像，将模板中的因果角色和成立条件映射到具体系统对象，生成符合 ChaosBlade 规范的故障场景，并结合实验结果判断目标机制是否被激活、预期后果是否出现以及结论是否具有充分证据。

主要挑战在于如何跨越系统和技术栈差异建立稳定的映射关系，以及如何表达无法判断、执行器不支持或证据不足等非确定性结果。需要围绕系统画像、缺陷模板、匹配结果、故障场景和

验证结论等核心概念完成对象建模，明确各对象的职责、关系、状态和接口；技术栈差异应通过可扩展的适配机制处理，使新增技术栈不会破坏已有匹配逻辑。要求基于平台现有技术栈实现可独立部署的服务，通过公开接口读取和写入平台数据，生成可由故障调度服务直接执行的实验配置。使用一个目标系统和不少于 5 条缺陷模板完成验证，比较自动匹配结果与人工判断，并报告匹配准确性、无法判定情况及实验结论。至少完成一组包含修改前后对照的实验，用于说明模块能否区分缺陷存在、未被触发和已被修复等不同情况。

4.2　参考链接与实验环境

参考事故报告链接：

1. AWS DynamoDB，2015-09-20 —— 元数据服务过载后重试放大，成员表请求超时又触发更多重试。
https://aws.amazon.com/message/5467D2/
2. AWS Kinesis，2020-11-25 —— 扩容使前端机线程数越过操作系统上限，缓存重建全量失败
https://aws.amazon.com/message/11201/
3. Roblox，2021-10-28 至 10-31 —— Consul 流式接口与 BoltDB 写放大互相推高负载，持续 73 小时。
https://corp.roblox.com/newsroom/2022/01/roblox-return-to-service-10-28-10-31-2021
4. Discord 语音，2026-03-25 —— 积压使单次处理成本上升，成本上升又加剧积压。
https://discord.com/blog/behind-the-scenes-of-the-3-25-26-voice-outage
5. Cloudflare，2019-07-02 —— 新增 WAF 规则中的正则回溯耗尽边缘 CPU。
https://blog.cloudflare.com/details-of-the-cloudflare-outage-on-july-2-2019/
6. Cloudflare，2025-11-18 —— 机器学习特征文件体积越过预设上限，边缘代理拒绝服务。
https://blog.cloudflare.com/18-november-2025-outage/
7. Google Cloud，2025-06-12 —— 配额策略数据含非预期空字段，全球分发后使配额检查进程崩溃。
https://status.cloud.google.com/incidents/ow5i3PPK96RduMcb1SsW
8. Fastly，2021-06-08 —— 客户配置激活了此前部署中的潜伏缺陷。
https://www.fastly.com/blog/summary-of-june-8-outage
9. Cloudflare，2026-02-20 —— 见原文。
https://blog.cloudflare.com/cloudflare-outage-february-20-2026
10. AWS S3，2017-02-28 —— 运维命令参数输入错误，摘除的服务器数量远超预期。
https://aws.amazon.com/message/41926/
11. AWS us-east-1，2021-12-07 —— 自动扩容触发内部网络客户端异常行为，造成内网拥塞。
https://aws.amazon.com/message/12721/
12. Google Cloud，2019-06-02 —— 配置错误叠加软件缺陷，自动化同时下线多个物理位置的独立集群。
https://status.cloud.google.com/incident/cloud-networking/19009
13. Datadog，2023-03-08 —— 操作系统组件自动更新在多区域同时生效，节点网络中断。
https://www.datadoghq.com/blog/2023-03-08-multiregion-infrastructure-connectivity-issue/
14. GitHub 可用性月报，2026 年 7 月 —— 一份月报含多起独立事故，适合做同月对比。
https://github.blog/news-insights/company-news/github-availability-report-july-2026/
15. GitHub，2018-10-21 —— 网络分区后 MySQL 拓扑切换导致跨站点写入分叉。
https://github.blog/2018-10-30-oct21-post-incident-analysis/
16. GitLab，2017-01-31 —— 误删生产数据目录，五种备份手段均未生效。
https://about.gitlab.com/blog/2017/02/10/postmortem-of-database-outage-of-january-31/

17. Monzo，2019-07-29 —— Cassandra 扩容期间的分片与一致性问题。
https://monzo.com/blog/2019/09/08/why-monzo-wasnt-working-on-july-29th
18. Atlassian，2022 年 4 月 —— 站点被误删，恢复流程本身成为持续两周的瓶颈。
https://www.atlassian.com/engineering/post-incident-review-april-2022-outage
19. AWS DynamoDB，2025-10-19 —— DNS 管理系统潜伏竞态产生空记录，自动修复机制无法自愈。
https://aws.amazon.com/message/101925/
20. Cloudflare，2026-01-22 —— 路由泄漏。
https://blog.cloudflare.com/route-leak-incident-january-22-2026/

实验系统访问地址：http://122.112.193.220:30881/

实现系统仓库地址：https://github.com/open-telemetry/opentelemetry-demo.git

实验环境凭证：ips：122.112.193.220、root/HociJePxS2DYnI6hsNjbFdqs